

ST/CS/MS/20/173

## Single Dimensional Array in Java

### Syntax to Declare an Array in Java

dataType[] arr; (or)  
dataType []arr; (or)  
dataType arr[];

### Instantiation of an Array in Java

```
arrayRefVar=new dataType[size];
```

### Example of Java Array

Let's see the simple example of java array, where we are going to declare, instantiate, initialize and traverse an array.

```
public class Testarray{  
    public static void main(String args[]){  
        int a[]=new int[5]; //declaration and instantiation  
        a[0]=1; //initialization  
        a[1]=2;  
        a[2]=7;  
        a[3]=4;  
        a[4]=5;  
        //traversing array  
        for(int i=0;i<a.length;i++)//length is the property of array  
            System.out.println(a[i]);  
    }  
}
```

### Output

|   |  |  |  |  |
|---|--|--|--|--|
| 1 |  |  |  |  |
| 2 |  |  |  |  |
| 7 |  |  |  |  |
| 4 |  |  |  |  |
| 5 |  |  |  |  |

### Declaration, Instantiation and Initialization of Java Array

We can declare, instantiate and initialize the java array together by:

```
int a[]={3,3,4,5}; //declaration, instantiation and initialization
```

## ArrayIndexOutOfBoundsException

The Java Virtual Machine (JVM) throws an `ArrayIndexOutOfBoundsException` if length of the array is negative, equal to the array size or greater than the array size while traversing the array.

```
public class TestArrayException {  
    public static void main(String args[]) {  
        int arr[]={50,60,70,80};  
        for(int i=0;i<=arr.length;i++){  
            System.out.println(arr[i]);  
        }  
    }  
}
```

Output

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 4  
at TestArrayException.main(TestArrayException.java:7)

```
50  
60  
70  
80
```

## Multidimensional Array in Java

In such case, data is stored in row and column based index (also known as matrix form).

### Syntax to Declare Multidimensional Array in Java

```
dataType[][] arrayRefVar; (or)  
dataType [][] arrayRefVar; (or)  
dataType arrayRefVar[][]; (or)  
dataType []arrayRefVar[];
```

### Example to instantiate Multidimensional Array in Java

```
int[][] arr=new int[3][3]; //3 row and 3 column
```

### Example to initialize Multidimensional Array in Java

```
arr[0][0]=1;  
arr[0][1]=2;  
arr[0][2]=3;  
arr[1][0]=4;  
arr[1][1]=5;  
arr[1][2]=6;  
arr[2][0]=7;
```

```

arr[2][1]=8;
arr[2][2]=9;
public class Testarray3{
    public static void main(String args[]){
        //declaring and initializing 2D array
        int arr[][]={{1,2,3},{2,4,5},{4,4,5}};
        //printing 2D array
        for(int i=0;i<3;i++){
            for(int j=0;j<3;j++){
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();
        }
    }
}

```

Output

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 4 | 5 |
| 4 | 4 | 5 |

Addition of 2 Matrices in Java

Let's see a simple example that adds two matrices.

```

public class Testarray5{
    public static void main(String args[]){
        //creating two matrices
        int a[][]={{1,3,4},{3,4,5}};
        int b[][]={{1,3,4},{3,4,5}};
        //creating another matrix to store the sum of two matrices
        int c[][]=new int[2][3];
        //adding and printing addition of 2 matrices
        for(int i=0;i<2;i++){
            for(int j=0;j<3;j++){
                c[i][j]=a[i][j]+b[i][j];
                System.out.print(c[i][j]+" ");
            }
            System.out.println();//new line
        }
    }
}

```

## Output

|   |   |    |
|---|---|----|
| 2 | 6 | 8  |
| 6 | 8 | 10 |

## Multiplication of 2 Matrices in Java

In the case of matrix multiplication, a one-row element of the first matrix is multiplied by all the columns of the second matrix which can be understood by the image given below.

$$\begin{array}{c}
 \text{Matrix 1} \left\{ \begin{array}{ccc} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{array} \right\}
 \end{array}
 \quad
 \begin{array}{c}
 \text{Matrix 2} \left\{ \begin{array}{ccc} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{array} \right\}
 \end{array}$$

$$\begin{array}{c}
 \text{Matrix 1} \\
 \star \\
 \text{Matrix 2}
 \end{array}
 \left\{ \begin{array}{ccc} 1*1+1*2+1*3 & 1*1+1*2+1*3 & 1*1+1*2+1*3 \\ 2*1+2*2+2*3 & 2*1+2*2+2*3 & 2*1+2*2+2*3 \\ 3*1+3*2+3*3 & 3*1+3*2+3*3 & 3*1+3*2+3*3 \end{array} \right\}$$

$$\begin{array}{c}
 \text{Matrix 1} \\
 \star \\
 \text{Matrix 2}
 \end{array}
 \left\{ \begin{array}{ccc} 6 & 6 & 6 \\ 12 & 12 & 12 \\ 18 & 18 & 18 \end{array} \right\}$$

```

public class MatrixMultiplicationExample{
    public static void main(String args[]){
        //creating two matrices
        int a[][]={{1,1,1},{2,2,2},{3,3,3}};
        int b[][]={{1,1,1},{2,2,2},{3,3,3}};

        //creating another matrix to store the multiplication of two matrices
        int c[][]=new int[3][3]; //3 rows and 3 columns

        //multiplying and printing multiplication of 2 matrices
        for(int i=0;i<3;i++){
            for(int j=0;j<3;j++){
                c[i][j]=0;
                for(int k=0;k<3;k++){
                    c[i][j]+=a[i][k]*b[k][j];
                }
            }
        }
    }
}

```

```

    } //end of k loop
    System.out.print(c[i][j]+" "); //printing matrix element
  } //end of j loop
  System.out.println(); //new line
}

```

```

}
}

```

Output

```

6 6 6
12 12 12
18 18 18

```

### Event-Driven Programming

A conventional model of computation has the program prescribe the exact order of input. Programs terminate once the input is exhausted.

Event-driven programs do not control the sequence in which input events occur.

### Event Handling

An event is a notification that something specific has occurred, such as a mouse click on a graphical button.

The event handler is a segment of code that is executed in response to an event.

### Event Sources

Input to an event-driven program comes from autonomous event sources.

### Event Properties

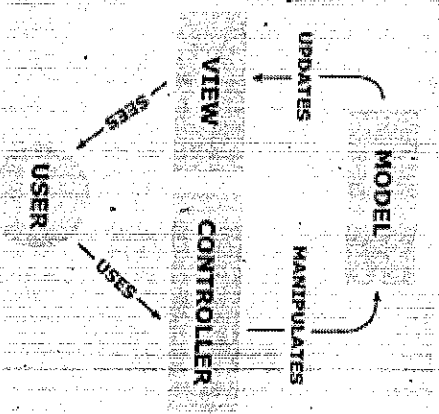
1. An event-driven program has no perceived stopping point.
2. The traditional read-eval-print loop does not explicitly appear.
3. An application processes an input and exits.

### Model-View-Controller (MVC)

Model: the object being implemented. Eg: game, calculator.

Controller: input mechanisms. Eg: buttons, menus, combo boxes.

View: output.



## Java GUI Application

A GUI application is a program that runs in its own window and communicates with users using buttons, menus, mouse clicks, etc.

A GUI application often has a paint method, which is invoked whenever the application needs to repaint itself.

## JavaFX vs Swing and AWT

Swing and AWT are replaced by the JavaFX platform for developing rich Internet applications.

When Java was introduced, the GUI classes were bundled in a library known as the Abstract Windows Toolkit (AWT). AWT is fine for developing simple graphical user interfaces, but not for developing comprehensive GUI projects. In addition, AWT is prone to platform-specific bugs. The AWT user-interface components were replaced by a more robust, versatile, and flexible library known as Swing components. Swing components are painted directly on canvases using Java code. Swing components depend less on the target platform and use less of the native GUI resources. Swing is designed for developing desktop GUI applications. It is now replaced by a completely new GUI platform known as JavaFX. JavaFX incorporates modern GUI technologies to enable you to develop rich Internet applications. A rich Internet application (RIA) is a Web application designed to deliver the same features and functions normally associated with desktop applications. A JavaFX application can run seamlessly on a desktop and from a Web browser. Additionally, JavaFX provides a multi-touch support for touch-enabled devices such as tablets and smart phones. JavaFX has a built-in 2D, 3D, animation support, video, and audio playback, and runs as a stand-alone application or from a browser. We decide to use Java GUI programming using JavaFX for two reasons. First, JavaFX is much simpler to learn and use for new Java programmers. Second, Swing is essentially dead, because it will not receive any further enhancement. JavaFX is the new GUI tool for developing cross-platform-rich Internet applications on desktop computers, on hand-held devices, and on the Web.

The Basic Structure of a JavaFX Program  
The abstract `javafx.application.Application` class defines the essential framework for writing JavaFX programs.

```

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;

public class MyJavaFX extends Application {
    @Override // Override the start method in the Application class
    public void start(Stage primaryStage) {
        // Create a scene and place a button in the scene
        Button btnOK = new Button("OK");
        Scene scene = new Scene(btnOK, 200, 250);
        primaryStage.setTitle("MyJavaFX"); // Set the stage title
        primaryStage.setScene(scene); // Place the scene in the stage
        primaryStage.show(); // Display the stage
    }

    public static void main(String[] args) {
        Application.launch(args);
    }
}

```

## Inheritance In Java

**Java Inheritance** is a mechanism in which one class acquires the property of another class. In Java, when an "is-A" relationship exists between two classes, we use Inheritance. The parent class is called a super class and the inherited class is called a subclass. The keyword `extends` is used by the sub class to inherit the features of super class.

Inheritance is important since it leads to the reusability of code.

### Java Inheritance Syntax:

```

class subclass extends superclass
{
    //methods and fields
}

```

## Java Inheritance Example

Here is an example of inheritance in Java:

A JavaServer Pages component is a type of Java servlet that is designed to fulfill the role of a user interface for a Java web application. Web developers write JSPs as text files that combine HTML or XHTML code, XML elements, and embedded JSP actions and commands.

Using JSP, you can collect input from users through Webpage forms, present records from a database or another source, and create Webpages dynamically.

JSP tags can be used for a variety of purposes, such as retrieving information from a database or registering user preferences, accessing JavaBeans components, passing control between pages, and sharing information between requests, pages etc.

## JSP - Syntax

### The Scriptlet

A scriptlet can contain any number of JAVA language statements, variable or method declarations, or expressions that are valid in the page scripting language.

Following is the syntax of Scriptlet

```
<% code fragment %>
```

Any text, HTML tags, or JSP elements you write must be outside the scriptlet. Following is the simple and first example for JSP

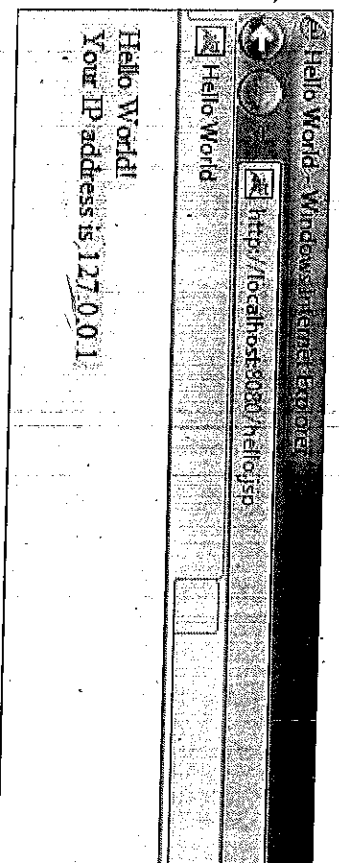
```
<html>
<head><title>Hello World</title></head>

<body>
  Hello World!<br/>
  <%
    out.println("Your IP address is " + request.getRemoteAddr());
  %>
</body>
</html>
```

**NOTE** – Assuming that Apache Tomcat is installed in C:\apache-tomcat-7.0.2 and your environment is setup as per environment setup.

Let us keep the above code in JSP file **hello.jsp** and put this file in C:\apache-tomcat7.0.2\webapps\ROOT directory. Browse through the same using URL <http://localhost:8080/hello.jsp>. The above code will generate the following result





## JSP Declarations

A declaration declares one or more variables or methods that you can use in Java code later in the JSP file. You must declare the variable or method before you use it in the JSP file.

Following is the syntax for JSP Declarations

```
<%! declaration; | declaration; | + ... %>
```

Following is an example for JSP Declarations

```
<%! int i = 0; %>
<%! int a, b, c; %>
<%! Circle a = new Circle(2.0); %>
```

## JSP Expression

A JSP expression element contains a scripting language expression that is evaluated, converted to a String, and inserted where the expression appears in the JSP file.

Because the value of an expression is converted to a String, you can use an expression within a line of text, whether or not it is tagged with HTML, in a JSP file.

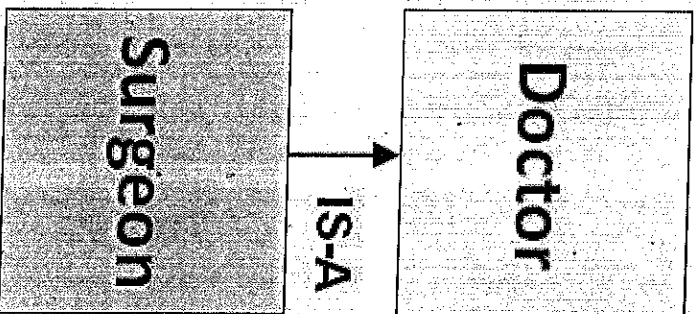
The expression element can contain any expression that is valid according to the Java Language Specification but you cannot use a semicolon to end an expression.

Following is the syntax of JSP Expression

```
<%= expression %>
```

Following example shows a JSP Expression

```
<html>
<head><title>A Comment Test</title></head>
<body>
<p>Today's date: <%= new java.util.Date().toLocaleString() %></p>
</html>
```



#### Java Inheritance Example

```
class Doctor {
    void Doctor_Details() {
        System.out.println("Doctor Details...");
    }
}

class Surgeon extends Doctor {
    void Surgeon_Details() {
        System.out.println("Surgeon Details...");
    }
}

public class Hospital {
    public static void main(String args[]) {
        Surgeon s = new Surgeon();
        s.Doctor_Details();
        s.Surgeon_Details();
    }
}
```

What is Encapsulation in Java?

**Encapsulation in Java** is a mechanism to wrap up variables(data) and methods(code) together as a single unit. It is the process of hiding information details and protecting data and behavior of the object. It is one of the four important OOP concepts. The encapsulate class is easy to test, so it is also better for unit testing.

## Encapsulation with an Example

To understand what is encapsulation in detail consider the following bank account class with deposit and show balance methods

```
class Account {
    private int account_number;
    private int account_balance;

    public void show Data() {
        //code to show data
    }

    public void deposit(int a) {
        if (a < 0) {
            //show error
        } else
            account_balance = account_balance + a;
    }
}
```

What is Polymorphism in Java?

**Polymorphism** in Java occurs when there are one or more classes or objects related to each other by inheritance. It is the ability of an object to take many forms. Inheritance lets users inherit attributes and methods, and polymorphism uses these methods to perform different tasks. So, the goal is communication, but the approach is different.

For example, you have a situation for communication. The communication mode you choose could be anything. It can be a call, a text message, a picture message, mail, etc. So, the goal is common that is communication, but their approach is different. This is called **Polymorphism**.

Polymorphism is the ability of an object to take on many forms. The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object.

Any Java object that can pass more than one IS-A test is considered to be polymorphic. In Java, all Java objects are polymorphic since any object will pass the IS-A test for their own type and for the class Object.

It is important to know that the only possible way to access an object is through a reference variable. A reference variable can be of only one type. Once declared, the type of a reference variable cannot be changed.

The reference variable can be reassigned to other objects provided that it is not declared final. The type of the reference variable would determine the methods that it can invoke on the object.

A reference variable can refer to any object of its declared type or any subtype of its declared type. A reference variable can be declared as a class or interface type.

### Example

```
public interface Vegetarian {}
```

```
public class Animal {}  
public class Deer extends Animal implements Vegetarian {}
```

Now, the Deer class is considered to be polymorphic since this has multiple inheritance. Following are true for the above examples –

- A Deer IS-A Animal
- A Deer IS-A Vegetarian
- A Deer IS-A Deer
- A Deer IS-A Object.

When we apply the reference variable facts to a Deer object reference, the following declarations are legal

#### Example

```
Deer d = new Deer();  
Animal a = d;  
Vegetarian v = d;  
Object o = d;
```

All the reference variables d, a, v, o refer to the same Deer object in the heap.

#### Overriding

If a class inherits a method from its superclass, then there is a chance to override the method provided that it is not marked final.

The benefit of overriding is: ability to define a behavior that's specific to the subclass type, which means a subclass can implement a parent class method based on its requirement.

In object-oriented terms, overriding means to override the functionality of an existing method.

#### Virtual Methods

In Virtual Method, We will see how the behavior of overridden methods in Java allows you to take advantage of polymorphism when designing your classes.

We already have discussed method overriding, where a child class can override a method in its parent. An overridden method is essentially hidden in the parent class, and is not invoked unless the child class uses the super keyword within the overriding method.

#### Example

```
public class Employee {  
    private String name;  
    private String address;  
    private int number;  
  
    public Employee(String name, String address, int number) {  
        System.out.println("Constructing an Employee");  
        this.name = name;  
        this.address = address;  
    }  
}
```

```

        this.number = number;
    }

    public void mailCheck() {
        System.out.println("Mailing a check to " + this.name + " " +
            this.address);
    }

    public String toString() {
        return name + " " + address + " " + number;
    }

    public String getName() {
        return name;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String newAddress) {
        address = newAddress;
    }

    public int getNumber() {
        return number;
    }
}

```

Now suppose we extend Employee class as follows

```

public class Salary extends Employee {
    private double salary; // Annual salary

    public Salary(String name, String address, int number, double salary) {
        super(name, address, number);
        setSalary(salary);
    }

    public void mailCheck() {
        System.out.println("Within mailCheck of Salary class ");
        System.out.println("Mailing check to " + getName()
            + " with salary " + salary);
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double newSalary) {
        if(newSalary >= 0.0) {
            salary = newSalary;
        }
    }

    public double computePay() {
        System.out.println("Computing salary pay for " + getName());
        return salary/52;
    }
}

```